Laboratorio 2: Modelado y análisis de datos con SQL

Valentina López, Isabella Montenegro, Lina Guarín
vlopezh@academia.usbbog.edu.co, iemontenegrom@academia.usbbog.edu.co, lkguarinc@academia.usbbog.edu.co
Facultad de Ingeniería
Bases de Datos Multimedia
Universidad de San Buenaventura
Bogotá, Colombia

***Resumen*—Este trabajo presenta el diseño, implementación y análisis de una base de datos relacional a partir de un conjunto de 999 registros de streamers. Inicialmente, se realizó la selección de los atributos relevantes para los requerimientos del laboratorio y se diseñó un modelo entidad-relación normalizado hasta la tercera forma normal (3FN). Posteriormente, el esquema se implementó en MySQL y se desarrolló un programa en Node.js para la lectura, transformación y carga de los datos, incorporando mecanismos de validación y manejo de errores. La integridad de la información se verificó mediante consultas de control y comprobaciones de las relaciones entre entidades. Finalmente, se ejecutaron ocho consultas SQL, cuatro básicas y cuatro analíticas, orientadas a estudiar los videojuegos transmitidos, los idiomas, los seguidores y la actividad semanal. Entre los hallazgos, Just Chatting destacó tanto como opción principal como secundaria, y el español registró el mayor promedio de seguidores por streamer. El estudio muestra la utilidad del modelo relacional para consultar datos y establecer comparaciones descriptivas, sin atribuir causalidad a las asociaciones observadas.**

***Index Terms*—Bases de datos relacionales, normalización, MySQL, Node.js, análisis de datos.**

I. Introducción

El crecimiento de las plataformas de transmisión en directo ha generado grandes cantidades de información relacionada con los creadores de contenido, sus audiencias y sus hábitos de actividad. El análisis de estos datos permite identificar tendencias y comparar características de los streamers mediante consultas estructuradas.

Sin embargo, para realizar estos análisis de manera organizada, es necesario contar con una estructura de almacenamiento que facilite la consulta de los datos y reduzca la redundancia de información.

En este contexto, el presente laboratorio tiene como objetivo diseñar e implementar una base de datos relacional a partir del archivo Streamers.csv, aplicando principios de modelado entidad-relación y normalización hasta la tercera forma normal, fundamentados en los conceptos de diseño de bases de datos relacionales descritos en [1].

El desarrollo comprende la selección de atributos, la construcción del esquema en MySQL y la implementación de un programa de carga mediante Node.js. Posteriormente, se utilizan consultas SQL para analizar las relaciones entre videojuegos, idiomas, seguidores y niveles de actividad de los streamers.

De esta manera, el trabajo integra el diseño conceptual, la implementación técnica y el análisis de resultados para demostrar la aplicación de los principios de bases de datos relacionales.

II. Selección, modelado y normalización de los datos

II-A. Selección y exclusión de información

El archivo Streamers.csv contiene 17 columnas relacionadas con las características, actividad y audiencia de los streamers. La selección de información se realizó considerando las cuatro preguntas analíticas asignadas y los requerimientos de las consultas SQL. Se priorizaron los atributos necesarios para identificar streamers, relacionarlos con sus videojuegos e idiomas y analizar sus métricas de audiencia y actividad, además de una variable complementaria para caracterizar la diversidad de contenido.

Se conservaron nueve columnas: NAME, LANGUAGE, MOST_STREAMED_GAME, 2ND_MOST_STREAMED_GAME, AVG_VIEWERS_PER_STREAM, TOTAL_FOLLOWERS, TOTAL_VIEWS, TOTAL_GAMES_STREAMED y ACTIVE_DAYS_PER_WEEK. NAME permite identificar a cada streamer; LANGUAGE facilita comparar la cantidad de streamers y el promedio de seguidores por idioma. MOST_STREAMED_GAME y 2ND_MOST_STREAMED_GAME permiten analizar la popularidad de los videojuegos principales y secundarios, así como identificar relaciones entre streamers. AVG_VIEWERS_PER_STREAM, TOTAL_FOLLOWERS y TOTAL_VIEWS proporcionan las métricas necesarias para comparar audiencias. Por último, ACTIVE_DAYS_PER_WEEK permite agrupar a los streamers según su frecuencia de actividad. TOTAL_GAMES_STREAMED se conservó como una medida complementaria de diversidad de contenido: indica cuántos juegos ha transmitido cada streamer, mientras que los campos principal y secundario identifican títulos concretos. Aunque no interviene directamente en las ocho consultas seleccionadas, permite ampliar el análisis de la actividad sin modificar el esquema.

Las ocho columnas restantes se excluyeron de acuerdo con los siguientes criterios: RANK se descartó porque las clasificaciones requeridas pueden obtenerse mediante consultas SQL utilizando ORDER BY, sin almacenar una posición que podría cambiar según el criterio de análisis. TYPE se excluyó porque las preguntas asignadas no requieren clasificar los streamers según su tipo. AVERAGE_STREAM_DURATION y TOTAL_TIME_STREAMED se descartaron porque los análisis solicitados no requieren medir la duración de las transmisiones ni el tiempo acumulado de emisión. FOLLOWERS_GAINED_PER_STREAM se excluyó porque el análisis de seguidores se centra en TOTAL_FOLLOWERS, sin requerir el crecimiento promedio por transmisión. AVG_GAMES_PER_STREAM se descartó porque las preguntas relacionadas con videojuegos requieren identificar y comparar juegos principales y secundarios, no calcular el promedio de juegos por transmisión. MOST_ACTIVE_DAY se excluyó porque el análisis de actividad utiliza la cantidad de días activos por semana, sin distinguir un día específico. DAY_WITH_MOST_FOLLOWERS_GAINED se descartó porque ninguna de las preguntas analíticas requiere identificar el día con mayor crecimiento de seguidores. La selección se centró en los atributos requeridos para las consultas del laboratorio. Se mantuvo además TOTAL_GAMES_STREAMED como excepción deliberada, por su utilidad para describir la diversidad de contenido y permitir análisis posteriores. Las demás exclusiones responden al alcance de las preguntas asignadas.

II-B. Entidades y relaciones

A partir de las nueve columnas seleccionadas del dataset se definieron cuatro tablas: STREAMER, LANGUAGUE, GAME y STREAMER_GAME. Esta organización permite separar las características de los streamers de los catálogos de idiomas y videojuegos.

STREAMER almacena el identificador, nombre, métricas de audiencia, seguidores, visualizaciones, cantidad de juegos transmitidos y días activos por semana. Su clave primaria es streamer_id y contiene language_id como clave foránea.

LANGUAGUE almacena los idiomas mediante languague_id como clave primaria y languague_name como atributo descriptivo. Entre LANGUAGUE y STREAMER existe una relación uno a muchos (1:N), debido a que varios streamers pueden compartir un mismo idioma, mientras que cada streamer pertenece a un único idioma registrado.

GAME contiene los videojuegos identificados mediante game_id como clave primaria y game_name como atributo descriptivo.

Entre STREAMER y GAME existe una relación muchos a muchos (N:M), ya que un streamer puede tener asociados distintos videojuegos y un mismo videojuego puede aparecer en varios streamers.

Esta relación se resuelve mediante STREAMER_GAME, que contiene streamer_id y game_id como claves foráneas, además de game_role, que identifica si el videojuego corresponde a la opción principal o secundaria.

La clave primaria compuesta por streamer_id, game_id y game_role permite distinguir las asociaciones incluso cuando un mismo videojuego aparece en ambas posiciones para un streamer.

De esta manera, el modelo representa las relaciones del dataset y reduce la duplicación de información descriptiva.

II-C. Normalización

La verificación de las tres formas normales se realizó a partir de las dependencias funcionales y las claves definidas para el esquema, siguiendo los principios de normalización relacional descritos en [1].

II-C1. Primera Forma Normal (1FN): El archivo original contiene valores atómicos en sus columnas, por lo que no se identifican celdas con listas de valores que incumplan directamente la 1FN. Sin embargo, MOST_STREAMED_GAME y 2ND_MOST_STREAMED_GAME representan dos posiciones de una misma relación entre streamers y videojuegos. Mantenerlas como columnas independientes limita la representación de dicha relación.

Por esta razón, se decidió representar las asociaciones mediante STREAMER_GAME, donde cada registro identifica un streamer, un videojuego y su función como juego principal o secundario mediante game_role. Así, cada asociación se almacena individualmente y se evita mantener columnas diferentes para cada posición de juego. Esta transformación responde a una decisión de diseño relacional y no a una falta de atomicidad en el CSV original.

II-C2. Segunda Forma Normal (2FN): Una relación cumple la 2FN cuando se encuentra en 1FN y sus atributos no clave no presentan dependencias parciales respecto de una clave candidata compuesta.

Las tablas STREAMER, LANGUAGUE y GAME utilizan claves primarias simples, por lo que no presentan dependencias parciales de una clave primaria compuesta. En STREAMER_GAME se definió la clave primaria compuesta por streamer_id, game_id y game_role. Esta tabla no contiene atributos descriptivos adicionales que dependan solamente de una parte de dicha clave: sus tres columnas forman la clave primaria y las dos primeras también funcionan como claves foráneas.

Por tanto, no se requiere una descomposición adicional para satisfacer la 2FN. La comprobación permite conservar la estructura planteada sin introducir tablas innecesarias.

II-C3. Tercera Forma Normal (3FN): Para verificar la 3FN se revisó que los atributos no clave dependieran de las claves de sus respectivas relaciones y que no existieran dependencias transitivas entre atributos no clave.

En STREAMER, las métricas de audiencia, seguidores, visualizaciones, juegos transmitidos y días activos se asocian directamente con streamer_id. El idioma se representa mediante language_id, que referencia la tabla LANGUAGUE, donde se almacena su nombre. De forma equivalente, GAME almacena los nombres de los videojuegos y STREAMER_GAME mantiene las referencias necesarias para relacionarlos con los streamers.

Esta separación centraliza los valores de idiomas y videojuegos, reduce su repetición y facilita mantener la integridad referencial. Aunque el CSV original no permite demostrar por sí solo una dependencia transitiva para todos estos campos, la revisión del esquema final no identifica dependencias transitivas entre atributos no clave. En consecuencia, el modelo propuesto satisface la 3FN.

II-D. Modelo entidad-relación final

El modelo entidad-relación final está compuesto por cuatro tablas: STREAMER, LANGUAGUE, GAME y STREAMER_GAME, como se presenta en la Fig. 1.

El esquema incorpora claves primarias para identificar los registros, claves foráneas para establecer las relaciones y una tabla asociativa que resuelve la relación muchos a muchos entre streamers y videojuegos.

La relación entre LANGUAGUE y STREAMER se implementa mediante language_id, mientras que STREAMER_GAME utiliza streamer_id y game_id para vincular streamers y videojuegos. El atributo game_role permite distinguir el juego principal del secundario.

El diseño resultante satisface las tres formas normales analizadas y constituye la base para la implementación del esquema mediante sentencias DDL en MySQL.

La correspondencia entre las entidades, sus atributos y las restricciones implementadas se desarrolla en la siguiente sección.

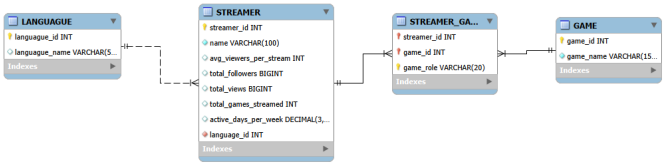


Figura 1. Modelo entidad-relación final.

III. Implementación del esquema (DDL)

III-A. Del modelo a las sentencias DDL

El script DDL se generó con la función *Forward Engineering* de MySQL Workbench a partir del modelo ER final (Fig. 1). Crea el esquema `strem` con conjunto de caracteres `utf8` y cuatro tablas que usan el motor InnoDB, el cual soporta transacciones y claves foráneas [2]. La Tabla I resume la correspondencia entre el modelo y las tablas creadas.

La relación entre `STREAMER` y `GAME` es de muchos a muchos (un streamer tiene un juego principal y puede tener otro secundario, y un juego es transmitido por muchos streamers), por lo que se resuelve con la tabla puente `STREAMER_GAME` [1]. La relación entre `LANGUAGUE` y `STREAMER` es de uno a muchos y se implementa con la clave foránea `language_id`. El identificador `LANGUAGUE` se conservó con esa grafía, tal como aparece en el modelo, para que el DDL corresponda exactamente a él.

III-B. Decisiones de tipos de datos y restricciones

Las decisiones se contrastaron con los valores observados en el CSV:

- **Claves y unicidad.** Las tablas STREAMER, LANGUAGUE y GAME utilizan claves primarias autoincrementales y restricciones UNIQUE sobre sus atributos de nombre. STREAMER_GAME utiliza una clave primaria compuesta por streamer_id, game_id y game_role.
- **El rol dentro de la clave primaria.** `game_role` forma parte de la clave de `STREAMER_GAME` porque un mismo juego puede ser principal y secundario de un streamer: en 5 registros del CSV el texto coincide (6 si se ignoran mayúsculas). Sin el rol en la clave, esas filas se considerarían duplicadas.
- **Numéricos.** Los máximos observados son 19 000 000 seguidores, 572 000 000 vistas, 481 615 espectadores promedio y 2 980 juegos transmitidos. Aunque los seguidores y las visualizaciones observados caben en INT, se eligió BIGINT para estas métricas acumuladas a fin de dejar margen de crecimiento [3]. Los espectadores promedio y la cantidad de juegos se almacenaron en INT.
- **Días activos por semana.** Se usó DECIMAL(3,1): es un tipo de punto fijo exacto [4], adecuado para valores con un decimal entre 0,0 y 7,0, que es el rango observado.
- **Longitudes de texto.** El nombre más largo observado tiene 24 caracteres (columna de 100), el juego 57 (columna de 150) y el idioma 10 (columna de 50), por lo que ningún valor excede su columna.
- **Acciones referenciales.** Las claves foráneas usan NO ACTION, que impide borrar o modificar una fila padre referenciada [5].
- **Collation.** El esquema utiliza `utf8` (alias de `utf8mb3` en MySQL 8). La *collation* de la base empleada no distingue mayúsculas [6]: “Valorant” y “VALORANT”, así como “TrackMania” y “Trackmania”, se consideran el mismo juego (Sección VII).

III-C. Fragmento del DDL

La Fig. 2 muestra la creación del esquema y las dos tablas con claves foráneas (se omiten los acentos graves y los índices de apoyo por brevedad; el script completo se conserva como `ddl.sql` en los archivos del proyecto).

IV. Programa de carga

IV-A. Flujo general

El programa se escribió en Node.js con la librería `mysql2` [7]. Lee el CSV, valida y transforma cada fila y la inserta en las cuatro tablas respetando el orden que imponen las claves foráneas (Fig. 3). Un segundo script, `verify.js`, consulta la base de datos para comprobar la carga (Sección VII).

IV-B. Selección de columnas

El programa usa 9 de las 17 columnas del archivo; las demás se descartaron según la justificación del análisis del dataset. La Tabla II muestra el destino de cada columna usada.

Tabla I
CORRESPONDENCIA ENTRE EL MODELO NORMALIZADO Y EL DDL

Tabla	Clave primaria	Claves foráneas	Restricciones y atributos
LANGUAGUE	languague_id (INT, AUTO_INCREMENT)	—	languague_name VARCHAR(50), UNIQUE
GAME	game_id (INT, AUTO_INCREMENT)	—	game_name VARCHAR(150) NOT NULL, UNIQUE
STREAMER	streamer_id (INT, AUTO_INCREMENT)	language_id → LANGUAGUE	name VARCHAR(100) NOT NULL, UNIQUE; avg_viewers_per_stream INT; total_followers y total_views BIGINT; total_games_streamed INT; active_days_per_week DECIMAL(3,1)
STREAMER_GAME	(streamer_id, game_id, game_role)	streamer_id → STREAMER; game_id → GAME	game_role VARCHAR(20) NOT NULL; ON DELETE/UPDATE NO ACTION

Tabla II
COLUMNAS DEL CSV USADAS Y SU DESTINO

Columna del CSV	Destino
NAME	STREAMER.name
LANGUAGE	LANGUAGUE.languague_name; FK STREAMER.language_id
MOST_STREAMED_GAME	GAME.game_name; fila en STREAMER_GAME con rol primary
2ND_MOST_STREAMED_GAME	GAME.game_name; fila en STREAMER_GAME con rol secondary
AVG_VIEWERS_PER_STREAM	STREAMER.avg_viewers_per_stream
TOTAL_FOLLOWERS	STREAMER.total_followers
TOTAL_VIEWS	STREAMER.total_views
TOTAL_GAMES_STREAMED	STREAMER.total_games_streamed
ACTIVE_DAYS_PER_WEEK	STREAMER.active_days_per_week

```
CREATE SCHEMA IF NOT EXISTS strem
  DEFAULT CHARACTER SET utf8;

CREATE TABLE IF NOT EXISTS strem.STREAMER (
  streamer_id INT NOT NULL AUTO_INCREMENT,
  name VARCHAR(100) NOT NULL,
  avg_viewers_per_stream INT NULL,
  total_followers BIGINT NULL,
  total_views BIGINT NULL,
  total_games_streamed INT NULL,
  active_days_per_week DECIMAL(3,1) NULL,
  language_id INT NOT NULL,
  PRIMARY KEY (streamer_id),
  UNIQUE INDEX name_UNIQUE (name),
  CONSTRAINT fk_STREAMER_LANGUAGUE
    FOREIGN KEY (language_id)
    REFERENCES strem.LANGUAGUE (languague_id)
    ON DELETE NO ACTION ON UPDATE NO ACTION
) ENGINE = InnoDB;

CREATE TABLE IF NOT EXISTS strem.STREAMER_GAME (
  streamer_id INT NOT NULL,
  game_id INT NOT NULL,
  game_role VARCHAR(20) NOT NULL,
  PRIMARY KEY (streamer_id, game_id, game_role),
  CONSTRAINT fk_STREAMER_GAME_STREAMER1
    FOREIGN KEY (streamer_id)
    REFERENCES strem.STREAMER (streamer_id),
  CONSTRAINT fk_STREAMER_GAME_GAME1
    FOREIGN KEY (game_id)
    REFERENCES strem.GAME (game_id)
) ENGINE = InnoDB;
```

Figura 2. Fragmento del DDL: esquema y tablas con claves foráneas.

IV-C. Transformaciones

Antes de insertar, cada fila pasa por funciones de validación que:

- eliminan espacios sobrantes de los textos y verifican que no estén vacíos ni excedan la longitud de la columna;
- convierten los campos numéricos, exigen enteros donde corresponde y validan que los días activos estén entre 0 y 7;
- tratan el juego secundario vacío como un dato válido: 77 de los 999 streamers no lo tienen, por lo que solo se crea su fila con rol `primary`.

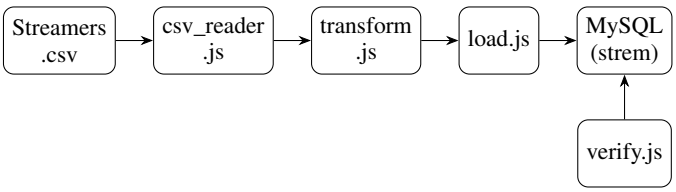


Figura 3. Flujo del programa de carga y de verificación.

Los nombres de idioma y de juego se insertan una sola vez; el programa guarda en memoria el identificador obtenido para reutilizarlo en las filas siguientes.

IV-D. Integridad durante la inserción

Las sentencias son parametrizadas (`execute` de `mysql2`), de modo que los valores viajan separados del SQL [7]. Toda la carga ocurre dentro de una transacción y cada fila se protege con un *savepoint*: si MySQL rechaza alguna inserción, se revierte solo esa fila con `ROLLBACK TO SAVEPOINT` y la carga continúa [8]. Esto evita dejar un streamer sin su juego principal o con relaciones a medias.

Tabla III
ERRORES PREVISIBLES Y RESPUESTA DEL PROGRAMA

Error	Respuesta
Tipo incompatible, valor vacío o fuera de rango (texto en un número, días > 7)	Se omite la fila y se registra como `INVALID_DATA`.
Fila con distinto número de columnas que el encabezado	Se omite y se registra como `INVALID_DATA`.
Violación de UNIQUE (streamer repetido)	MySQL responde `ER_DUP_ENTRY`; se revierte la fila con *savepoint*, se registra y se continúa.
Violación de clave foránea u otro error de la base	Igual que el caso anterior, con el código de error de MySQL.
Juego o idioma que la *collation* considera igual a otro	Se reutiliza el identificador existente y se deja un aviso en el resumen.
Columna requerida ausente en el archivo	Se detiene la carga con un mensaje claro.
Error inesperado o de conexión	*Rollback* de la transacción, mensaje en consola y código de salida 1.

Tabla IV
REGISTROS POR TABLA TRAS LA CARGA

Tabla	Filas
`STREAMER`	999
`LANGUAGUE`	19
`GAME`	211
`STREAMER_GAME`	1921
rol `primary`	999
rol `secondary`	922

Tabla V
CHEQUEOS DE INTEGRIDAD (RESULTADO ESPERADO: 0)

Chequeo	Resultado
Streamers sin idioma asociado	0
Streamers sin juego principal	0
Juegos que ningún streamer usa	0
Días activos fuera del rango 0–7	0

V. DECISIÓN ARQUITECTÓNICA

Se optó por un **script independiente** (`load.js` apoyado en `csv_reader.js` y `transform.js`) en lugar de la arquitectura por capas del proyecto (rutas, controladores y servicios). Las razones son:

1. La carga es un proceso por lotes que se ejecuta una vez sobre una base vacía y no atiende peticiones de usuarios; las capas web solo añadirían archivos sin aportar funcionalidad.
2. Requiere un flujo secuencial: el orden lo imponen las claves foráneas (idiomas y juegos, luego streamers, luego relaciones) y el manejo de errores por fila con *savepoints* se controla mejor en un solo flujo.
3. Se conserva la modularidad: la validación está en funciones puras (`transform.js`) que pueden probarse sin base de datos, la lectura del CSV en su propio módulo y el acceso a MySQL en `load.js`.
4. El procedimiento puede verificarse: `node load.js` realiza la carga inicial sobre una base vacía y `node verify.js` contrasta los conteos y comprueba la integridad de las relaciones. Una nueva carga sin limpiar previamente la base no es idempotente, como se documenta en la Sección VIII.

El script reutiliza el archivo de configuración de conexión del proyecto (`env/mysqlConfig.js`). Si más adelante la carga debiera dispararse desde una interfaz web, la arquitectura por capas sería preferible; para este laboratorio resulta innecesaria.

VI. MANEJO DE ERRORES

La Tabla III resume los errores previsibles y la reacción del programa.

Registro de errores. Al terminar, el programa imprime un resumen (filas leídas, insertadas, rechazadas y avisos), guarda el detalle de cada fila rechazada en un archivo JSON y finaliza con código de salida distinto de cero si hubo errores [9]. Es una aplicación del principio de *fail fast*: los problemas se hacen visibles de inmediato en lugar de ocultarse [10].

VI-A. Pruebas del manejo de errores

Prueba 1: repetir la carga. Se ejecutó de nuevo el programa sobre la base ya cargada. Las 999 filas fueron rechazadas por la restricción UNIQUE de `STREAMER.name`: el programa lo reportó en su resumen (`errors: 999`, Fig. 7), registró cada fila con el código `ER_DUP_ENTRY` (Fig. 8) y la base conservó sus conteos e integridad (Fig. 9).

VII. VERIFICACIÓN DE LA CARGA

El script `verify.js` consulta directamente la base `strem` y realiza tres comprobaciones: el conteo de filas por tabla, cuatro chequeos de integridad referencial y la comparación con el resumen que produjo el programa de carga (Figs. 4 y 5).

La Tabla IV presenta los conteos obtenidos tras una carga limpia. Son consistentes con el CSV: hay un registro principal por cada uno de los 999 streamers y uno secundario por cada streamer que lo tiene ($999 - 77 = 922$), lo que da $999 + 922 = 1921$ filas en `STREAMER_GAME`. Una consulta directa en MySQL, `SELECT COUNT(*) FROM strem.STREAMER`, confirmó los 999 streamers (Fig. 6).

Los cuatro chequeos de integridad dieron 0 (Tabla V) y los conteos de la base coincidieron con los del programa de carga (Fig. 5).

Juegos: 211 y no 213. El CSV contiene 213 nombres de juego distintos si se comparan exactamente, pero solo 211 si se ignoran las mayúsculas. La base registró 211 porque la *collation* considera iguales "Valorant" y "VALORANT" y también "TrackMania" y "Trackmania" [6]. El programa detectó ambos casos, reutilizó el registro existente y lo informó como aviso en el resumen de la carga (Fig. 4).

VIII. Nota sobre la versión del programa entregado

Después de la prueba de repetir la carga se propuso una revisión menor del programa: (i) no sobrescribir el resumen de una carga exitosa cuando una nueva ejecución no inserta nada y (ii) emitir el aviso de identificador reutilizado solo cuando los nombres difieren. Esa revisión **no se incorporó**: el programa entregado es la versión previa. Esta decisión se consultó con el docente, quien indicó que esa versión es adecuada para los fines del laboratorio.

La consecuencia es que, si se repite la carga sobre una base ya poblada, el archivo de resumen queda con ceros y `verify.js` reporta diferencias (líneas *FAIL* en la Fig. 9) aunque los datos sigan intactos; además, el resumen incluye avisos repetidos de idiomas cuyo nombre es idéntico al existente (Fig. 7). Para obtener la evidencia final se eliminó la base, se volvió a crear con el DDL y se ejecutó la carga una sola vez. Esto no afecta la corrección de los datos cargados.

IX. Evidencias

Las figuras siguientes documentan la ejecución del programa y sus resultados de verificación. Como respaldo adicional, el proyecto incluye los archivos `load_summary_Streamers.json` y `load_errors_Streamers.json`, que registran los resultados de la carga y los errores controlados.

El código fuente, el modelo entidad relación, el script DDL y las ocho consultas SQL se encuentran disponibles en el repositorio de GitHub del proyecto: https://github.com/BuuValu/LAb2-BasesDeDatos

IX-A. Carga exitosa y verificación

Las Figs. 4–6 corresponden a la carga limpia sobre la base recién creada con el DDL.

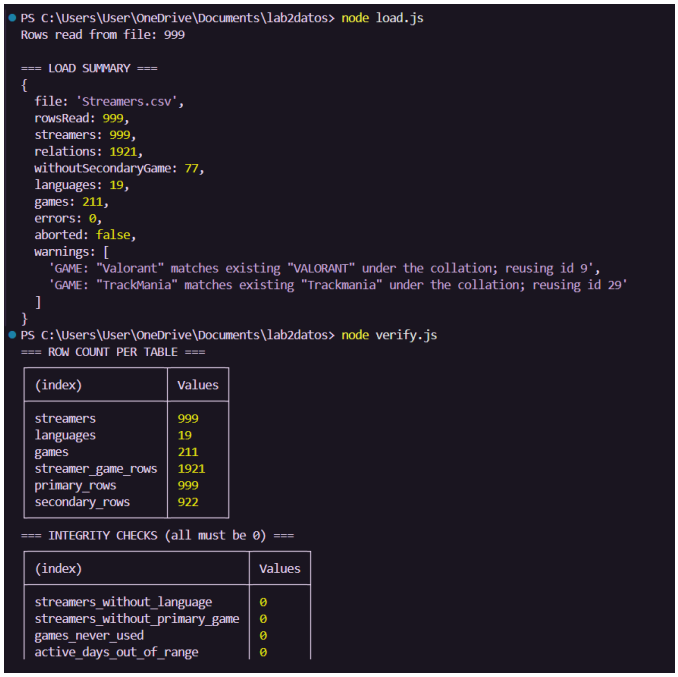


Figura 4. Resumen de `node load.js`: 999 filas leídas, 999 streamers, 19 idiomas, 211 juegos y 1921 relaciones insertados, 77 streamers sin juego secundario y `errors: 0`. Los dos avisos corresponden a "Valorant"/"VALORANT" y "TrackMania"/"Trackmania", que la *collation* trata como iguales. Abajo se inicia `node verify.js` con los conteos por tabla y los chequeos de integridad en 0.

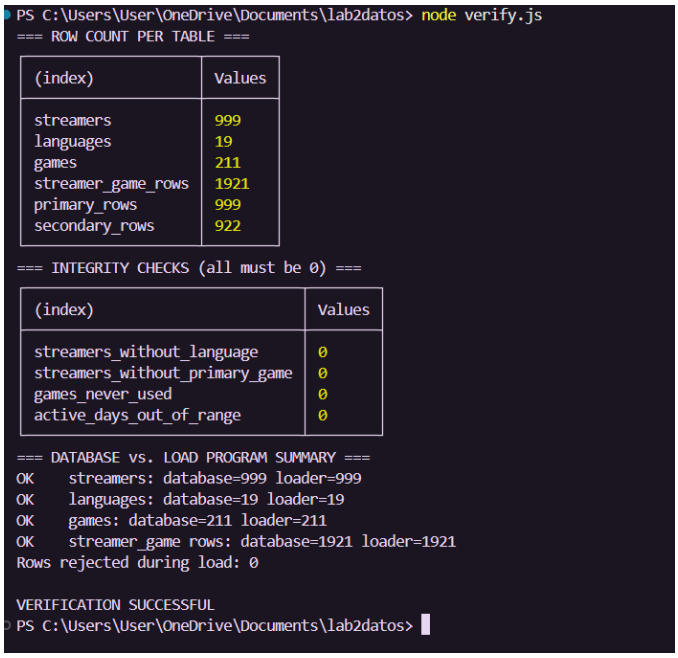


Figura 5. Salida completa de `node verify.js` tras la carga limpia: conteos por tabla (999, 19, 211, 1921; 999 primarios y 922 secundarios), los cuatro chequeos de integridad en 0 y la comparación base de datos contra programa de carga con OK en las cuatro líneas. Termina con *VERIFICATION SUCCESSFUL*.

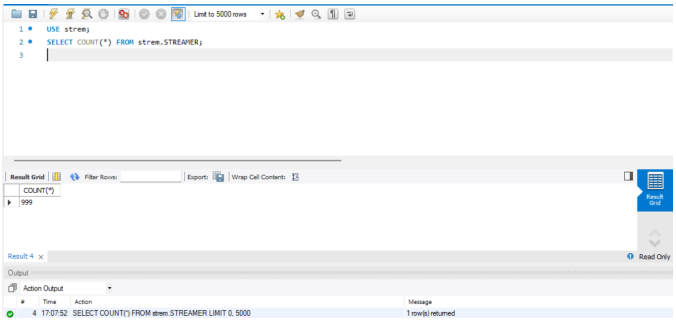


Figura 6. Consulta directa en MySQL Workbench: `SELECT COUNT(*) FROM strem.STREAMER` devuelve 999, lo que confirma la carga independientemente del programa Node.js.

IX-B. Rechazo de datos repetidos

Las Figs. 7–9 corresponden a la Prueba 1 (repetir la carga sobre la base ya poblada).

```
PS C:\Users\User\OneDrive\Documents\lab2datos> node load.js
Rows read from file: 999

=== LOAD SUMMARY ===
{
  file: 'streamers.csv',
  rowsRead: 999,
  streamers: 0,
  relations: 0,
  withoutSecondaryGame: 0,
  languages: 0,
  games: 0,
  errors: 999,
  aborted: false,
  warnings: [
    'LANGUAGUE: "English" matches existing "English" under the collation; reusing id 1',
    'LANGUAGUE: "English" matches existing "English" under the collation; reusing id 1',
    'LANGUAGUE: "English" matches existing "English" under the collation; reusing id 1',
    'LANGUAGUE: "English" matches existing "English" under the collation; reusing id 1',
    'LANGUAGUE: "Spanish" matches existing "Spanish" under the collation; reusing id 2',
    'LANGUAGUE: "Spanish" matches existing "Spanish" under the collation; reusing id 2',
    'LANGUAGUE: "French" matches existing "French" under the collation; reusing id 3',
    'LANGUAGUE: "English" matches existing "English" under the collation; reusing id 1',
    'LANGUAGUE: "English" matches existing "English" under the collation; reusing id 1',
    'LANGUAGUE: "German" matches existing "German" under the collation; reusing id 4',
    'LANGUAGUE: "Russian" matches existing "Russian" under the collation; reusing id 5',
    'LANGUAGUE: "French" matches existing "French" under the collation; reusing id 3',
    'LANGUAGUE: "Japanese" matches existing "Japanese" under the collation; reusing id 6',
    'LANGUAGUE: "Japanese" matches existing "Japanese" under the collation; reusing id 6',
    'LANGUAGUE: "Spanish" matches existing "Spanish" under the collation; reusing id 2',
    'LANGUAGUE: "English" matches existing "English" under the collation; reusing id 1',
    'LANGUAGUE: "German" matches existing "German" under the collation; reusing id 4',
    'LANGUAGUE: "English" matches existing "English" under the collation; reusing id 1',
    'LANGUAGUE: "Chinese" matches existing "Chinese" under the collation; reusing id 7',
    'LANGUAGUE: "English" matches existing "English" under the collation; reusing id 1',
    'LANGUAGUE: "English" matches existing "English" under the collation; reusing id 1',
    'LANGUAGUE: "Portuguese" matches existing "Portuguese" under the collation; reusing id 8',
    'LANGUAGUE: "Japanese" matches existing "Japanese" under the collation; reusing id 6',
    'LANGUAGUE: "Portuguese" matches existing "Portuguese" under the collation; reusing id 8',
```

Figura 7. Resumen de la segunda ejecución de `node load.js`: de 999 filas leídas se insertaron 0 y se rechazaron 999 (`errors: 999`, `aborted: false`). Los avisos de idioma reutilizado se explican en la Sección VIII.

```
    ... 899 more items
  ]
}

999 row(s) rejected. See load_errors_Streamers.json
```

(index)	line	name	type	detail
0	2	'kaicenat'	'ER_DUP_ENTRY'	"Duplicate entry 'kaicenat' for key 'streamer.name_UNIQUE'"
1	3	'jynxzi'	'ER_DUP_ENTRY'	"Duplicate entry 'jynxzi' for key 'streamer.name_UNIQUE'"
2	4	'caedrel'	'ER_DUP_ENTRY'	"Duplicate entry 'caedrel' for key 'streamer.name_UNIQUE'"
3	5	'caseoh_'	'ER_DUP_ENTRY'	"Duplicate entry 'caseoh_' for key 'streamer.name_UNIQUE'"
4	6	'ibai'	'ER_DUP_ENTRY'	"Duplicate entry 'ibai' for key 'streamer.name_UNIQUE'"
5	7	'auronplay'	'ER_DUP_ENTRY'	"Duplicate entry 'auronplay' for key 'streamer.name_UNIQUE'"
6	8	'zerator'	'ER_DUP_ENTRY'	"Duplicate entry 'zerator' for key 'streamer.name_UNIQUE'"
7	9	'tarik'	'ER_DUP_ENTRY'	"Duplicate entry 'tarik' for key 'streamer.name_UNIQUE'"
8	10	'riotgames'	'ER_DUP_ENTRY'	"Duplicate entry 'riotgames' for key 'streamer.name_UNIQUE'"
9	11	'papaplatte'	'ER_DUP_ENTRY'	"Duplicate entry 'papaplatte' for key 'streamer.name_UNIQUE'"

Figura 8. Detalle de las filas rechazadas: línea del CSV, nombre del streamer, tipo de error `ER_DUP_ENTRY` y el mensaje de MySQL por violación de la restricción `streamer.name_UNIQUE`. El detalle completo se guarda en `load_errors_Streamers.json`.

```
PS C:\Users\User\OneDrive\Documents\lab2datos> node verify.js
=== ROW COUNT PER TABLE ===
```

(index)	Values
streamers	999
languages	19
games	211
streamer_game_rows	1921
primary_rows	999
secondary_rows	922

```
=== INTEGRITY CHECKS (all must be 0) ===
```

(index)	Values
streamers_without_language	0
streamers_without_primary_game	0
games_never_used	0
active_days_out_of_range	0

```
=== DATABASE vs. LOAD PROGRAM SUMMARY ===
FAIL  streamers: database=999 loader=0
FAIL  languages: database=19 loader=0
FAIL  games: database=211 loader=0
FAIL  streamer_game rows: database=1921 loader=0
Rows rejected during load: 999

VERIFICATION FAILED
PS C:\Users\User\OneDrive\Documents\lab2datos>
```

Figura 9. `node verify.js` tras la carga repetida: la base conserva sus conteos (999, 19, 211, 1921) y los chequeos de integridad siguen en 0, es decir, no se corrompió ni duplicó ningún dato. Las líneas FAIL se deben a que el resumen del programa quedó en 0 (Sección VIII); por eso se rehizo la carga limpia de las Figs. 4–5.

X. Consultas SQL y análisis de resultados

Una vez implementado el modelo relacional y verificada la carga de los 999 streamers, se ejecutaron ocho consultas SQL sobre la base de datos strem.

Las primeras cuatro consultas permiten comprobar la aplicación de filtros, relaciones entre entidades y funciones de agregación. Las cuatro restantes responden las preguntas analíticas establecidas para el dataset Streamers.csv.

Para cada consulta se presenta su objetivo, la sentencia SQL utilizada, los resultados obtenidos y su respectiva interpretación.

X-A. Consultas básicas

X-A1. Filtrado mediante WHERE: La primera consulta tuvo como objetivo identificar los diez streamers con mayor cantidad de seguidores entre aquellos que registran al menos un millón de seguidores y cuatro o más días activos por semana. Para ello, se utilizaron dos condiciones mediante WHERE y AND, ordenando los resultados de forma descendente.

```
SELECT
    name,
    total_followers,
    active_days_per_week
FROM STREAMER
WHERE total_followers >= 1000000
  AND active_days_per_week >= 4
ORDER BY total_followers DESC
LIMIT 10;
```

La Fig. 10 presenta los diez registros obtenidos mediante la ejecución de la consulta.

Result Grid | Filter Rows: | Export:

name	total_followers	active_days_per_week
ninja	19000000	4.1
auronplay	16300000	5.0
ibai	15600000	4.3
rubius	15000000	5.0
xqc	12000000	6.3
thegrefg	11900000	6.3
juansguarnizo	11200000	5.5
shroud	10900000	5.4
elmariana	9610000	4.2
pokimane	9320000	4.4

Figura 10. Streamers con mayor cantidad de seguidores que cumplen las condiciones de actividad establecidas.

Como se observa en la Fig. 10, Ninja encabeza los resultados con 19 millones de seguidores y 4,1 días activos por semana. Le siguen Auronplay, con 16,3 millones de seguidores y 5 días activos, e Ibai, con 15,6 millones y 4,3 días.

Aunque todos los streamers seleccionados cumplen las condiciones establecidas, presentan diferencias considerables en sus cantidades de seguidores. Por ejemplo, Ninja tiene más seguidores que otros streamers con una mayor frecuencia de actividad semanal.

X-A2. Relación entre dos entidades mediante JOIN: La segunda consulta buscó identificar los diez streamers registrados con idioma español que presentan mayor cantidad de seguidores. Para ello, se relacionaron las tablas STREAMER y LANGUAGE mediante INNER JOIN, utilizando la clave foránea language_id.

```
SELECT
    s.name,
    l.language_name AS idioma,
    s.total_followers
FROM STREAMER AS s
INNER JOIN LANGUAGUE AS l
    ON s.language_id = l.language_id
WHERE l.language_name = 'Spanish'
ORDER BY s.total_followers DESC
LIMIT 10;
```

La Fig. 11 presenta los diez streamers de idioma español con mayor cantidad de seguidores.

Los resultados de la Fig. 11 identifican a Auronplay como el streamer de idioma español con mayor cantidad de seguidores, con 16,3 millones. Le siguen Ibai, con 15,6 millones, y Rubius, con 15 millones. En la décima posición se encuentra Quackitytoo, con 4,62 millones.

La diferencia entre el primer y el décimo puesto es considerable: los 16,3 millones de Auronplay equivalen a aproximadamente 3,5 veces los 4,62 millones de Quackitytoo dentro de este listado.

X-A3. Relación entre tres entidades: La tercera consulta tuvo como objetivo identificar los diez streamers con mayor promedio de espectadores por transmisión y determinar cuál es su videojuego principal.

Para obtener esta información se relacionaron las tablas

Result Grid | Filter Rows:

name	idioma	total_followers
auronplay	Spanish	16300000
ibai	Spanish	15600000
rubius	Spanish	15000000
thegrefg	Spanish	11900000
juansguarnizo	Spanish	11200000
elmariana	Spanish	9610000
elspreen	Spanish	9240000
rivers_gg	Spanish	6060000
elded	Spanish	5580000
quackitytoo	Spanish	4620000

Figura 11. Resultados de la consulta JOIN entre STREAMER y LANGUAGUE.

STREAMER, STREAMER_GAME y GAME mediante dos operaciones INNER JOIN. Adicionalmente, se filtraron las asociaciones cuyo atributo game_role corresponde a primary.

```
SELECT
    s.name AS streamer,
    g.game_name AS juego_principal,
    s.avg_viewers_per_stream
        AS espectadores_promedio
FROM STREAMER AS s
INNER JOIN STREAMER_GAME AS sg
    ON s.streamer_id = sg.streamer_id
INNER JOIN GAME AS g
    ON sg.game_id = g.game_id
WHERE sg.game_role = 'primary'
ORDER BY s.avg_viewers_per_stream DESC
LIMIT 10;
```

La Fig. 12 muestra los streamers seleccionados, sus videojuegos principales y el promedio de espectadores por transmisión.

Result Grid | Filter Rows: | Export:

streamer	juego_principal	espectadores_promedio
twitchrivals	League of Legends	481615
ow_esports	Overwatch	400009
gcxevent	Destiny 2	387406
fortnite	Fortnite	352166
rocketleague	Rocket League	349645
riotgames	League of Legends	346968
evo	Special Events	338908
att	Counter-Strike	261664
echo_esports	World of Warcraft	249671
gaules	Counter-Strike	235565

Figura 12. Streamers con mayor promedio de espectadores y sus videojuegos principales.

Los resultados de la Fig. 12 muestran que TwitchRivals ocupa la primera posición con 481.615 espectadores promedio, asociado con League of Legends. Le siguen OW Esports, con 400.009 espectadores y Overwatch como juego principal, y GCXEvent, con 387.406 espectadores y Destiny 2.

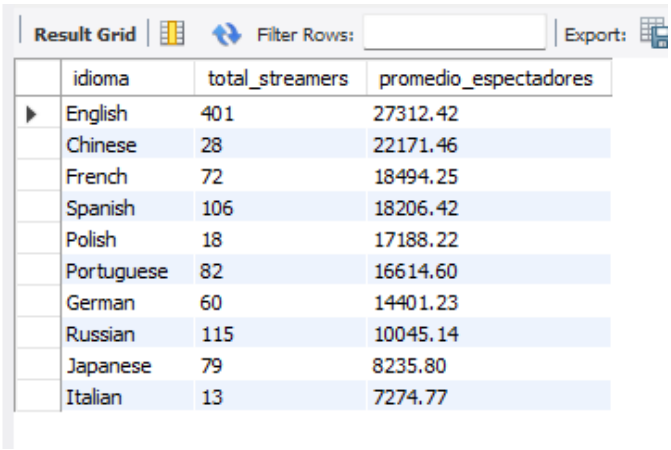
Asimismo, League of Legends y Counter-Strike aparecen dos veces entre los diez registros seleccionados. El resultado describe su presencia en este grupo, sin atribuir la audiencia únicamente al título transmitido.

X-A4. Agregación mediante GROUP BY: La cuarta consulta buscó determinar cuáles idiomas presentan un mayor promedio de espectadores por transmisión, considerando únicamente aquellos con al menos diez streamers registrados.

Para ello, se utilizaron las funciones COUNT y AVG, junto con GROUP BY para agrupar los registros por idioma. La cláusula HAVING permitió excluir los grupos con menos de diez streamers.

```
SELECT
    l.languague_name AS idioma,
    COUNT(s.streamer_id)
        AS total_streamers,
    ROUND(
        AVG(s.avg_viewers_per_stream), 2
    ) AS promedio_espectadores
FROM STREAMER AS s
INNER JOIN LANGUAGUE AS l
    ON s.language_id = l.languague_id
GROUP BY
    l.languague_id,
    l.languague_name
HAVING COUNT(s.streamer_id) >= 10
ORDER BY promedio_espectadores DESC;
```

La Fig. 13 presenta los resultados de la agrupación por idioma.



Result Grid | Filter Rows: | Export:

idioma	total_streamers	promedio_espectadores
English	401	27312.42
Chinese	28	22171.46
French	72	18494.25
Spanish	106	18206.42
Polish	18	17188.22
Portuguese	82	16614.60
German	60	14401.23
Russian	115	10045.14
Japanese	79	8235.80
Italian	13	7274.77

Figura 13. Cantidad de streamers y promedio de espectadores por idioma.

Como se observa en la Fig. 13, el inglés presenta el mayor promedio de espectadores, con 27.312,42 y un total de 401 streamers. Le siguen el chino, con 22.171,46 espectadores promedio, y el francés, con 18.494,25.

La comparación entre chino y español resulta ilustrativa: con 28 streamers, el chino registra 22.171,46 espectadores promedio; el español, con 106 registros, alcanza 18.206,42. El tamaño del grupo y su audiencia promedio no siguen el mismo orden.

X-B. Consultas de análisis

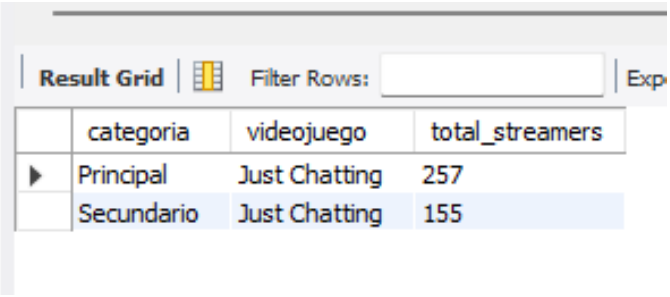
Las siguientes consultas responden las cuatro preguntas analíticas asignadas al dataset Streamers.csv. A diferencia de las consultas básicas, su propósito es establecer comparaciones entre variables y extraer conclusiones a partir de los resultados obtenidos.

X-B1. Juegos principales y secundarios: El primer análisis buscó determinar cuál es el videojuego que aparece con mayor frecuencia como opción principal entre los streamers y establecer si coincide con el más frecuente como segunda opción.

Para ello, se calcularon las frecuencias de cada videojuego según el atributo game_role y se identificó el valor máximo en ambas categorías.

```
WITH conteos AS (
    SELECT
        g.game_name,
        SUM(CASE
            WHEN sg.game_role = 'primary'
                THEN 1
            ELSE 0
        END) AS total_principal,
        SUM(CASE
            WHEN sg.game_role = 'secondary'
                THEN 1
            ELSE 0
        END) AS total_secundario
    FROM GAME AS g
    INNER JOIN STREAMER_GAME AS sg
        ON g.game_id = sg.game_id
    GROUP BY
        g.game_id,
        g.game_name
)
SELECT
    'Principal' AS categoria,
    game_name AS videojuego,
    total_principal AS total_streamers
FROM conteos
WHERE total_principal = (
    SELECT MAX(total_principal)
    FROM conteos
)
UNION ALL
SELECT
    'Secundario' AS categoria,
    game_name AS videojuego,
    total_secundario AS total_streamers
FROM conteos
WHERE total_secundario = (
    SELECT MAX(total_secundario)
    FROM conteos
);
```

La Fig. 14 presenta los contenidos más frecuentes en cada categoría.



Result Grid | Filter Rows: | Exp

categoria	videojuego	total_streamers
Principal	Just Chatting	257
Secundario	Just Chatting	155

Figura 14. Contenido más frecuente como juego principal y secundario.

Los resultados muestran que Just Chatting ocupa el primer lugar en ambas categorías. Aparece como contenido principal de 257 streamers y como contenido secundario de 155.

Por tanto, el contenido más frecuente como transmisión principal coincide con el más utilizado como segunda opción. Esto evidencia una presencia destacada de Just Chatting dentro del conjunto analizado, tanto como actividad principal como alternativa de transmisión.

Es importante señalar que Just Chatting corresponde a una categoría de contenido conversacional y no a un videojuego propiamente dicho. Sin embargo, se conserva en el análisis porque el dataset lo incluye dentro de los campos correspondientes a videojuegos transmitidos.

X-B2. Idiomas y promedio de seguidores: El segundo análisis tuvo como objetivo identificar el idioma con mayor promedio de seguidores totales por streamer y determinar si coincide con el idioma que concentra la mayor cantidad de registros.

Para ello, se agruparon los streamers por idioma y se calcularon tanto la cantidad de participantes como el promedio de seguidores mediante COUNT y AVG.

```
SELECT
    l.language_name AS idioma,
    COUNT(s.streamer_id)
        AS total_streamers,
    ROUND(
        AVG(s.total_followers), 2
    ) AS promedio_seguidores
FROM STREAMER AS s
INNER JOIN LANGUAGUE AS l
    ON s.language_id = l.languague_id
GROUP BY
    l.languague_id,
    l.languague_name
ORDER BY promedio_seguidores DESC;
```

La Fig. 15 presenta los promedios de seguidores y las cantidades de streamers correspondientes a los diferentes idiomas.

Los resultados muestran que el español presenta el mayor promedio de seguidores, con 1.776.153,77 por streamer y 106 participantes.

En segundo lugar se encuentra el inglés, con un promedio de 1.133.736,96 seguidores, aunque concentra la mayor cantidad de streamers del dataset, con 401 registros.

X-B3. Relaciones entre streamers según videojuegos: El tercer análisis buscó identificar streamers cuyo videojuego principal coincide con el videojuego secundario de otro streamer y comparar sus cantidades de seguidores y visualizaciones totales.

Para ello, se relacionó la tabla STREAMER_GAME consigo misma, utilizando las asociaciones de videojuegos principales y secundarios pertenecientes a streamers diferentes.

Se utilizó ROW_NUMBER para seleccionar una pareja representativa por videojuego, priorizando las mayores cantidades de seguidores, y se limitaron los resultados a diez coincidencias.

```
WITH coincidencias AS (
    SELECT
        g.game_name AS juego,
        s1.name AS streamer_principal,
        s1.total_followers
            AS seguidores_principal,
        s2.name AS streamer_secundario,
        s2.total_followers
            AS seguidores_secundario,
        s1.total_views AS vistas_principal,
        s2.total_views AS vistas_secundario,
        ROW_NUMBER() OVER (
            PARTITION BY g.game_id
            ORDER BY
                s1.total_followers DESC,
                s2.total_followers DESC
        ) AS posicion
    FROM STREAMER_GAME AS sg1
    INNER JOIN STREAMER_GAME AS sg2
        ON sg1.game_id = sg2.game_id
        AND sg1.streamer_id <> sg2.streamer_id
    INNER JOIN STREAMER AS s1
        ON sg1.streamer_id = s1.streamer_id
    INNER JOIN STREAMER AS s2
        ON sg2.streamer_id = s2.streamer_id
    INNER JOIN GAME AS g
        ON sg1.game_id = g.game_id
    WHERE sg1.game_role = 'primary'
      AND sg2.game_role = 'secondary'
)
SELECT
    juego,
    streamer_principal,
    seguidores_principal,
    streamer_secundario,
    seguidores_secundario,
    vistas_principal,
    vistas_secundario
FROM coincidencias
WHERE posicion = 1
ORDER BY seguidores_principal DESC
LIMIT 10;
```

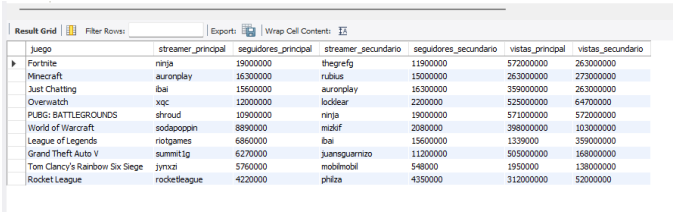
La Fig. 16 presenta diez ejemplos de coincidencias entre videojuegos principales y secundarios de distintos streamers.

Los resultados permiten identificar coincidencias en títulos como Fortnite, Minecraft, PUBG: BATTLEGROUNDS y Lea-

Result Grid | Filter Rows: | Export:

idioma	total_streamers	promedio_seguidores
Spanish	106	1776153.77
English	401	1133736.96
Turkish	7	1094714.29
Portuguese	82	775176.83
French	72	766026.39
Italian	13	740930.77
German	60	735735.00
Polish	18	588500.00
Korean	2	459500.00
Thai	5	455800.00
Russian	115	442713.19
Czech	3	343333.33
Japanese	79	272054.43
Cantonese	2	264500.00
Chinese	28	232975.00
Hungarian	1	229000.00
Ukrainian	3	205966.67
Arabic	1	160000.00
Romanian	1	141000.00

Figura 15. Promedio de seguidores y cantidad de streamers por idioma.



juego	streamer_principal	seguidores_principal	streamer_secundario	seguidores_secundario	vistas_principal	vistas_secundario
Fortnite	ninja	19000000	thegrefg	11900000	572000000	263000000
Minecraft	auronplay	16300000	rubius	15000000	263000000	273000000
Just Chatting	ibai	15600000	auronplay	16300000	359000000	263000000
Overwatch	xqc	12000000	locklear	2200000	525000000	64700000
PUBG: BATTLEGROUNDS	shroud	10900000	ninja	19000000	571000000	572000000
World of Warcraft	sodapoppin	8890000	mizkif	2080000	398000000	103000000
League of Legends	riotgames	6860000	ibai	15600000	1339000	359000000
Grand Theft Auto V	summit1g	6270000	juansguarnizo	11200000	505000000	168000000
Tom Clancy's Rainbow Six Siege	jynxzi	5760000	mobilmobil	548000	1950000	138000000
Rocket League	rocketleague	4220000	philza	4350000	312000000	52000000

Figura 16. Coincidencias entre videojuegos principales y secundarios de distintos streamers.

gue of Legends.

En Fortnite, Ninja tiene el juego como opción principal: registra 19 millones de seguidores y 572 millones de visualizaciones. TheGrefg lo tiene como secundario, con 11,9 millones de seguidores y 263 millones de visualizaciones.

En Minecraft, Auronplay figura con el juego como principal y Rubius como secundario. Aunque Auronplay supera a Rubius en seguidores (16,3 frente a 15 millones), Rubius acumula más visualizaciones (273 frente a 263 millones).

Estos ejemplos no muestran una jerarquía uniforme en seguidores y visualizaciones según la prioridad del juego.

Debe considerarse que la consulta presenta diez parejas representativas seleccionadas mediante criterios de ordenamiento. En consecuencia, estos ejemplos no permiten generalizar el comportamiento de todos los streamers.

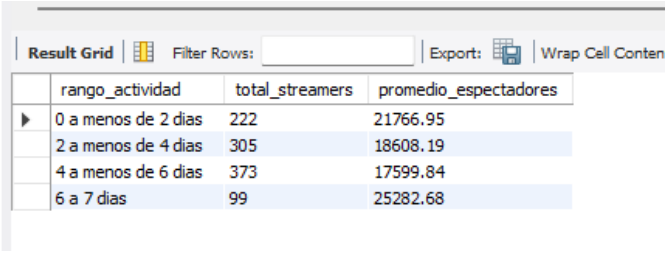
X-B4. Días activos y audiencia promedio: El cuarto análisis tuvo como objetivo determinar qué rango de días activos por semana presenta el mayor promedio de espectadores por transmisión y evaluar si una mayor actividad se relaciona necesariamente con una audiencia superior.

Para realizar la comparación, los streamers se clasificaron en cuatro rangos de actividad semanal: menos de dos días, de dos a menos de cuatro, de cuatro a menos de seis y de seis a siete días.

Posteriormente, se calcularon la cantidad de streamers y el promedio de espectadores de cada grupo.

```
WITH actividad AS (
    SELECT
        streamer_id,
        avg_viewers_per_stream,
        CASE
            WHEN active_days_per_week < 2
                THEN '0 a menos de 2 dias'
            WHEN active_days_per_week < 4
                THEN '2 a menos de 4 dias'
            WHEN active_days_per_week < 6
                THEN '4 a menos de 6 dias'
            ELSE '6 a 7 dias'
        END AS rango_actividad,
        CASE
            WHEN active_days_per_week < 2
                THEN 1
            WHEN active_days_per_week < 4
                THEN 2
            WHEN active_days_per_week < 6
                THEN 3
            ELSE 4
        END AS orden_rango
    FROM STREAMER
    WHERE active_days_per_week BETWEEN 0 AND 7
      AND avg_viewers_per_stream IS NOT NULL
)
SELECT
    rango_actividad,
    COUNT(streamer_id) AS total_streamers,
    ROUND(
        AVG(avg_viewers_per_stream), 2
    ) AS promedio_espectadores
FROM actividad
GROUP BY
    orden_rango,
    rango_actividad
ORDER BY orden_rango;
```

La Fig. 17 presenta la cantidad de streamers y el promedio de espectadores por transmisión para cada rango de actividad semanal.



rango_actividad	total_streamers	promedio_espectadores
0 a menos de 2 dias	222	21766.95
2 a menos de 4 dias	305	18608.19
4 a menos de 6 dias	373	17599.84
6 a 7 dias	99	25282.68

Figura 17. Promedio de espectadores por transmisión según el rango de días activos por semana.

El rango de seis a siete días activos obtiene el mayor promedio: 25.282,68 espectadores entre 99 streamers. El grupo de cuatro a menos de seis días, con 373 streamers, registra el menor promedio (17.599,84).

Los otros grupos comprenden 222 streamers con menos de dos días activos (21.766,95 espectadores promedio) y 305 entre dos y menos de cuatro días (18.608,19). Los promedios no aumentan de manera continua con la actividad.

Estas medias también pueden verse influidas por cuentas con audiencias excepcionalmente altas. Por ello, las diferencias observadas no deben atribuirse solo a los días activos.

El análisis utiliza AVG_VIEWERS_PER_STREAM como medida de audiencia promedio por transmisión, la cual debe distinguirse de TOTAL_VIEWS, que corresponde a visualizaciones acumuladas.

X-C. Discusión de resultados

Las consultas básicas verificaron que el esquema permite filtrar streamers, recuperar información de distintas entidades y resumir sus métricas por idioma. Esta capacidad depende de las relaciones entre STREAMER, LANGUAGUE, GAME y STREAMER_GAME, sin necesidad de reconstruir la tabla plana original.

En las consultas analíticas se observaron diferencias entre presencia y alcance. Just Chatting resultó ser el contenido más frecuente tanto como opción principal como secundaria, mientras que el español registró el mayor promedio de seguidores por streamer, pese a que el inglés concentra más registros. Las coincidencias de videojuegos entre streamers tampoco determinaron diferencias uniformes en seguidores o vistas.

Por último, los rangos de actividad semanal mostraron promedios de audiencia no monotónicos: el grupo de seis a siete días registró el promedio más alto, pero los grupos intermedios no siguieron una progresión creciente. Estas comparaciones

son descriptivas y están condicionadas por la composición del conjunto de datos, por lo que no permiten establecer causalidad.

XI. Conclusiones

La selección de nueve de las 17 columnas del archivo original, incluida una variable complementaria sobre diversidad de juegos, permitió construir un esquema relacional centrado en las preguntas del laboratorio. La separación de streamers, idiomas y videojuegos, junto con la tabla asociativa STREAMER_GAME, evitó repetir nombres de juegos e idiomas en cada registro y permitió justificar un modelo normalizado hasta la tercera forma normal.

La implementación con MySQL y Node.js cargó correctamente los 999 streamers, 19 idiomas, 211 juegos y 1.921 relaciones. Los chequeos de integridad devolvieron cero incidencias y la comparación entre los resultados de la base y el resumen del cargador fue satisfactoria. El manejo explícito de los 77 streamers sin juego secundario también evitó introducir relaciones ficticias.

Las consultas analíticas revelaron que Just Chatting es la opción más frecuente tanto en el rol principal (257 streamers) como en el secundario (155). Asimismo, el español alcanzó el mayor promedio de seguidores por streamer, aunque el inglés reunió la mayor cantidad de registros. El rango de seis a siete días activos presentó la audiencia promedio más alta, sin que exista una relación creciente y constante entre actividad y espectadores.

El trabajo confirma que la normalización y el uso de SQL facilitan comparaciones verificables sobre un mismo conjunto de datos. Como limitaciones, los resultados son descriptivos y el cargador no puede repetirse sobre una base ya poblada sin generar rechazos por duplicidad. Una mejora futura sería hacer idempotente el programa de carga y ampliar el análisis con las variables excluidas cuando surjan nuevas preguntas.

Referencias

[1] R. Elmasri and S. B. Navathe, *Fundamentals of Database Systems*, 7th ed. Pearson, 2016.

[2] Oracle Corporation, "The InnoDB storage engine," *MySQL 8.0 Reference Manual*. [Online]. Available: https://dev.mysql.com/doc/refman/8.0/en/innodb-storage-engine.html. Accessed: Oct. 10, 2026.

[3] Oracle Corporation, "Integer types (exact value) – INTEGER, INT, SMALLINT, TINYINT, MEDIUMINT, BIGINT," *MySQL 8.0 Reference Manual*. [Online]. Available: https://dev.mysql.com/doc/refman/8.0/en/integer-types.html. Accessed: Oct. 10, 2026.

[4] Oracle Corporation, "Fixed-point types (exact value) – DECIMAL, NUMERIC," *MySQL 8.0 Reference Manual*. [Online]. Available: https://dev.mysql.com/doc/refman/8.0/en/fixed-point-types.html. Accessed: Oct. 10, 2026.

[5] Oracle Corporation, "FOREIGN KEY constraints," *MySQL 8.0 Reference Manual*. [Online]. Available: https://dev.mysql.com/doc/refman/8.0/en/create-table-foreign-keys.html. Accessed: Oct. 10, 2026.

[6] Oracle Corporation, "Character sets, collations, Unicode," *MySQL 8.0 Reference Manual*. [Online]. Available: https://dev.mysql.com/doc/refman/8.0/en/charset.html. Accessed: Oct. 10, 2026.

[7] "MySQL2 documentation." [Online]. Available: https://sidorares.github.io/node-mysql2/docs. Accessed: Oct. 10, 2026.

[8] Oracle Corporation, "SAVEPOINT, ROLLBACK TO SAVEPOINT, and RELEASE SAVEPOINT statements," *MySQL 8.0 Reference Manual*. [Online]. Available: https://dev.mysql.com/doc/refman/8.0/en/savepoint.html. Accessed: Oct. 10, 2026.

[9] OpenJS Foundation, "Process – `process.exitCode`," *Node.js Documentation*. [Online]. Available: https://nodejs.org/api/process.html. Accessed: Oct. 10, 2026.

[10] J. Shore, "Fail fast," *IEEE Software*, vol. 21, no. 5, pp. 21–25, Sep./Oct. 2004.